

Exact Residual-Core Reduction for Multiway High-Dimensional Fixed-Effect Projection

Hypergraph leaf elimination for arbitrary categorical FE partitions

Anonymous technical manuscript

Implementation reference: `econhdfc/hdfc/numerical_core.py`

September 2026

Abstract

High-dimensional fixed-effect (HDFE) solvers repeatedly project outcome and regressor blocks onto the orthogonal complement of a large categorical dummy design. Existing graph-based HDFE work already recognizes degree-one pruning as a useful device in two-way fixed-effect networks. This note gives an exact arbitrary-multiway formulation for the projection problem. The categorical design is represented as a multipartite hypergraph whose observations are hyperedges and whose fixed-effect levels are vertices. With strictly positive observation weights, if an active fixed-effect level is incident to exactly one active observation, the first-order condition for that dummy coefficient forces the residual on that observation to be exactly zero. The observation can therefore be eliminated without altering the residuals on the remaining observations. Repeating the argument yields an exact residual core for any finite number of categorical fixed-effect partitions. The full HDFE residual is obtained by solving only on the core and inserting exact zeros on peeled rows. We prove the reduction for scalar and multiple right-hand sides, show why duplicate observation tuples cannot be collapsed in the projection problem, characterize the positive-weight requirement and zero-weight invalidation rule, and map each mathematical claim to the implementation and regression tests in `econhdfc`. The technical contribution is the arbitrary- G exact residual-core reduction and its solver-safe formulation; the underlying idea of graph leaf pruning is not claimed to be new.

Review and priority boundary (version 0.6.1). The package review independently checks the proof steps, assumptions and code correspondence, supplemented by exact finite examples and numerical oracles. This is not a machine-checked formal proof, exhaustive verification of all finite inputs, or a certification of historical priority. Established linear algebra, singleton elimination, functional dependencies and orthogonal reduction retain their prior-art status. The package’s framework and implementation may be useful without establishing a previously unknown theorem. See the accompanying `mathematical-review-0.6.1.md` for qualifications and regression evidence. **Keywords:** high-dimensional fixed effects; alternating projections; hypergraph; leaf elimination; weighted least squares; residualization; categorical projection.

JEL codes: C13, C63, C87.

Claim boundary. Degree-one pruning is already present in graph-based HDFE solver work, notably in the two-way graph formulation of Correia (2017) and the `reghdfc` documentation. The contribution here is narrower: an exact projection theorem and implementation for an arbitrary number of categorical FE partitions, expressed as a multiway hypergraph residual-core reduction. Adaptive MAP/CG routing, fused multi-RHS kernels, buffer reuse, threading, and memory pooling are engineering choices and are not claimed as technical innovations.

1 Projection problem

Let $D_g \in \{0, 1\}^{N \times K_g}$ denote the one-hot design for fixed-effect partition $g = 1, \dots, G$, and let

$$D = [D_1 \ D_2 \ \dots \ D_G].$$

For a right-hand side $y \in \mathbb{R}^N$ and strictly positive diagonal weight matrix $W = \text{diag}(w_1, \dots, w_N)$, $w_i > 0$, the weighted HDFE projection solves

$$\hat{\alpha} \in \arg \min_{\alpha} (y - D\alpha)'W(y - D\alpha), \quad r = y - D\hat{\alpha}. \quad (1)$$

The fitted coefficients need not be unique when D is rank deficient, but the fitted values $D\hat{\alpha}$ and residual vector r are unique because they are weighted orthogonal projections onto $\text{col}(D)$ and its complement.

For multiple right-hand sides $Y \in \mathbb{R}^{N \times M}$, the objective is the sum of (1) across columns. Every statement below then applies columnwise.

2 Hypergraph representation

Each observed FE level is a vertex. Each observation i is a hyperedge connecting the G vertices corresponding to its levels $(c_{1i}, \dots, c_{Gi})$. Unlike the exact-rank problem, repeated FE tuples are distinct hyperedges because they carry distinct values y_i , weights w_i , and possibly distinct columns of a multi-RHS block.

Definition 2.1 (Active degree). Given an active observation set $A \subseteq \{1, \dots, N\}$, the active degree of FE vertex v is the number of active observations incident to v .

Definition 2.2 (Residual core). Starting from all observations active, repeatedly remove an observation incident to any vertex of active degree one. The remaining observation set is the *residual core*. The process terminates when no active FE vertex has degree one.

The order of processing degree-one vertices need not be unique, but the set of observations surviving iterative degree-one elimination is the usual 2-core-like residual of the incidence hypergraph under leaf removal. The implementation uses a queue and updates vertex degrees in $O(G)$ time per peeled observation.

3 Exact leaf-elimination identity

Lemma 3.1 (Unique-level residual is zero). *Suppose FE level v occurs in exactly one active observation i . Under strictly positive weight $w_i > 0$, every weighted least-squares solution satisfies $r_i = 0$.*

Proof. Let α_v be the coefficient on dummy column v . The first-order condition is

$$0 = \frac{\partial}{\partial \alpha_v} (y - D\alpha)'W(y - D\alpha) = -2 \sum_{j: v \in j} w_j r_j.$$

By assumption the sum contains only observation i , so $w_i r_i = 0$. Since $w_i > 0$, $r_i = 0$. $\square$

Lemma 3.2 (Leaf elimination preserves the remaining optimization). *Under the conditions of Lemma 3.1, removing observation i and the unique coefficient α_v leaves the weighted least-squares problem for all other residuals unchanged.*

Proof. Write the linear predictor on row i as $\alpha_v + s_i(\gamma)$, where γ collects all coefficients except α_v . Because v appears nowhere else, for every fixed γ we may choose

$$\alpha_v = y_i - s_i(\gamma),$$

which sets row- i residual and loss contribution exactly to zero. Therefore

$$\min_{\alpha_v, \gamma} \sum_{j=1}^N w_j r_j^2 = \min_{\gamma} \sum_{j \neq i} w_j r_j^2.$$

The minimizing residuals for rows $j \neq i$ are exactly those of the reduced problem. $\square$

Theorem 3.3 (Exact recursive residual-core reduction). *Let C be the residual core obtained by recursively removing degree-one observations from an intercept-only categorical G -way FE design with strictly positive weights. Let r_C be the weighted HDFE residual obtained by solving the reduced problem on C , with FE codes restricted and redensified on C . Then the full-sample weighted residual is*

$$r_i = \begin{cases} (r_C)_i, & i \in C, \\ 0, & i \notin C. \end{cases}$$

The same statement holds simultaneously for any finite block of right-hand sides.

Proof. Apply Lemma 3.2 to the first peeled observation. The reduced problem has the same remaining residuals. Recompute active degrees; if another degree-one vertex appears, apply the lemma again. Induction over the finite peeling sequence removes all non-core rows. Each peeled row has residual zero by Lemma 3.1, while the final active rows solve the residual-core problem. Multi-RHS projection is separable across columns, so the same elimination sequence applies to all columns. $\square$

Corollary 3.4 (Exact orthogonality). *If the core solver returns residuals satisfying the weighted FE normal equations on the reduced core, the reconstructed full residual satisfies $D'Wr = 0$ exactly on peeled rows and to the numerical accuracy of the core solver on retained rows.*

4 Why projection peeling differs from structural-rank peeling

The exact structural DoF engine in `econhdfc/hdfc/rank.py` may deduplicate identical FE tuples because duplicate incidence rows do not increase matrix rank. Projection cannot do this: two observations with the same FE tuple can have different y_i , X_i , or weights. The residual-core algorithm therefore retains every observation as a separate hyperedge and removes rows only when the weighted first-order conditions prove their residuals are zero.

This distinction is important for software architecture. Structural-rank preprocessing changes an algebraic representation used to count the dimension of $\text{col}(D)$; residual-core reduction changes only the numerical problem used to compute the projection. Neither may alter the user-requested FE topology used for reporting and inference conventions.

5 Weights and topology lifecycle

Strictly positive weights are sufficient for Theorem 3.3. Their magnitudes do not affect the incidence topology or the implication $w_i r_i = 0$. Consequently, once the core topology is compiled, it can be reused across arbitrary positive weight updates.

Zero weights are different. If $w_i = 0$, a unique-level first-order condition becomes $0 \cdot r_i = 0$ and no longer forces $r_i = 0$. Moreover, zero-weight rows do not contribute to the weighted objective, so the effective projection topology is the support of positive weights. The implementation therefore invalidates/disables residual-core reuse when a weight update contains zeros rather than applying the positive-weight theorem outside its assumptions.

Negative or nonfinite weights are rejected before projection. Positive weights here are finite real numbers. Exact zero insertion on peeled rows is distinct from the finite numerical accuracy of the core solver; neither the theorem nor a small Krylov objective update alone guarantees that the requested residual tolerance has been attained.

6 Algorithm

For dense integer FE codes, the implementation stores one degree counter and one row-index sum per FE vertex. The row-index sum allows the unique incident observation to be recovered when the degree equals one. A queue is initialized with all degree-one vertices. When an observation is peeled, the degree and row-index sum of each of its G incident vertices are updated; newly degree-one vertices enter the queue.

1. Compute level offsets for the G partitions.
2. Accumulate degree and incident-row-index sums for all vertices.
3. Queue all vertices of degree one.
4. Pop a degree-one vertex; recover its unique active observation.
5. Mark the observation inactive and update all G incident vertices.
6. Repeat until the queue is empty.
7. Extract core rows and redensify each FE code vector on the core.
8. Solve the HDFE projection only on the core.
9. Reconstruct the full residual by inserting exact zeros on peeled rows.

If $V = \sum_g K_g$, topology compilation is $O(GN + V)$ time and $O(GN + V)$ memory in the current stacked-code implementation, with the queue/update phase itself $O(GN + V)$. A streaming construction can reduce temporary code-stacking memory but does not change the theorem.

7 Relationship to established HDFE solvers

Guimarães and Portugal (2010) and Gaure (2013) establish iterative projection methods for multiple HDFEs. Correia (2017) develops graph-based acceleration and explicitly includes pruning of low-degree vertices in the two-way graph representation; current `reghdfe` documentation also lists

degree-one pruning as a preconditioning option. The present result should therefore be read as a multiway exact generalization and a projection-specific proof, not as the invention of leaf pruning.

The theorem is independent of the solver used on the core. Symmetric MAP with conjugate-gradient acceleration, LSMR/LSQR, a specialized two-way Schur solver, or another exact-enough HDFE projection method may be used. The residual-core transformation changes the system size, not the statistical estimand.

8 Implementation correspondence

Mathematical object	Implementation / validation
Hypergraph leaf queue	<code>econhdf/hdf/numerical_core.py</code> <code>_peel_active_mask</code>
Core extraction/redensification	<code>build_numerical_core</code>
Positive-weight topology reuse	<code>HDFEAbsorber.update_weights</code> lifecycle
Zero-weight invalidation	residual-core disabled after zero-weight update
Full/core parity	<code>tests/test_hdf_multiway_solver_opt.py</code>
Weighted dense oracle	same test module, direct weighted dummy least squares
Exact zero on peeled rows	<code>test_core_plan_peeling_rows_</code> <code>are_exactly_zero_residuals</code>
Complex FE corpus	<code>tests/test_hdf_complex_fe_corpus.py</code>
Performance evidence	<code>benchmarks/hdf/solver_v044_integration.json</code>

The production implementation deliberately does not treat adaptive MAP/CG choice, fused weighted multi-RHS projection, or in-place output-buffer reuse as mathematical contributions. Those are execution optimizations layered around the exact reduction proved here.

9 Validation evidence

The audited implementation contains the following regression checks:

- parity between reduced and unreduced 3+ FE projection;
- parity against an explicit dense weighted dummy-design oracle on small systems;
- exact zero residuals on every peeled row;
- positive-weight topology reuse across weight updates;
- zero-weight invalidation;
- copy/in-place semantics so core reduction does not silently add a second full $N \times M$ result allocation;
- stable and mobility FE corpora with multiple interaction FE structures.

Performance gains depend on peelability. Leaf-rich designs can shrink dramatically, while irreducible mobility designs may have an empty peeled set and therefore receive no core-size benefit. This is an expected property, not a failure mode.

10 Limitations

The theorem is intentionally limited to intercept-only categorical FE blocks. Heterogeneous slopes create coefficient columns whose first-order conditions are weighted by continuous covariates; a degree-one categorical level need not imply the same elimination identity when the relevant term structure differs. Group-individual multi-membership FE designs also require a separate incidence model. Zero-weight observations invalidate the positive-weight topology argument. Finally, the theorem is an exact reduction of the projection problem, not a convergence theorem for the iterative solver used on the residual core.

11 Conclusion

For any finite number of intercept-only categorical fixed-effect partitions and strictly positive weights, recursive degree-one hypergraph peeling is an exact elimination procedure for HDFE residualization. Every peeled observation has zero projection residual, and the residuals on the surviving observations are exactly those obtained by solving the reduced core problem. The result provides a theorem-backed arbitrary-multiway counterpart to graph leaf pruning already known in two-way HDFE work, while preserving a strict separation between numerical execution topology and statistical inference topology.

References

- Correia, S. (2017). Linear models with high-dimensional fixed effects: An efficient and feasible estimator. Working paper. <https://scoreia.com/research/hdfe.pdf>.
- Correia, S. and Constantine, N. (2026). `reghdfe`: Stata module for linear and instrumental-variable/GMM regression absorbing multiple levels of fixed effects. <https://github.com/sergiocorreia/reghdfe>.
- Gaure, S. (2013). OLS with multiple high dimensional category variables. *Computational Statistics & Data Analysis*, 66:8–18.
- Guimarães, P. and Portugal, P. (2010). A simple feasible procedure to fit models with high-dimensional fixed effects. *The Stata Journal*, 10(4):628–649.